

UNIVERSIDAD DEL VALLE DE GUATEMALA

Colegio Universitario



Exámen parcial No. 2

Catedrático: Tomás Gálvez

Jose Pablo Ordoñez Barrios – 231329

Análisis y diseño de algoritmos

Sección 20

Guatemala, 2026

Enlace de GitHub: <https://github.com/JoseOrdonezB/parcial2-ADA.git>

Explicación de código: <https://youtu.be/x43YKWit0CQ>

Problema No. 1

Considere el problema de “hacer sencillo”: dado un monto m , buscamos alcanzar dicho monto con la menor cantidad de monedas posible. Las monedas tienen las siguientes denominaciones: {1,5,10,25}. Podemos tomar todas las monedas que necesitemos de cada denominación.

a. Propiedades que posee el problema.

- Subestructura óptima
- Subproblemas traslapados
- Greedy-choice property

b. Demostración de las propiedades.

Subestructura óptima:

Sea $OPT(m)$ una solución óptima para el monto m , supongamos que $OPT(m)$ utiliza una moneda de valor c . Entonces el resto de monedas debe resolver el subproblema $m - c$. Si la solución para $m - c$ no fuera óptima, podría reemplazarse por una mejor, obteniendo una solución mejor para m , lo cual es una contradicción. Por lo tanto, el problema posee subestructura óptima.

Subproblemas traslapados:

Para resolver $OPT(m)$ se calculan subproblemas como: $OPT(m - 1)$, $OPT(m - 5)$, $OPT(m - 10)$, $OPT(m - 25)$. Estos mismos subproblemas aparecen repetidamente al resolver otros montos. Por lo tanto, el problema posee subproblemas traslapados.

Greedy-choice property:

Se selecciona la moneda de mayor valor posible, con las monedas {1, 5, 10, 25}. Reemplazar varias monedas pequeñas por una moneda mayor nunca aumenta la cantidad de monedas usadas, por lo que al elegir la moneda de mayor valor posible siempre puede formar parte de una solución óptima. Por lo tanto, el problema posee greedy-choice property.

c. Pseudocódigo

hacerSencillo(m):

monedas = {1, 5, 10, 25}

cantidad = 0

for moneda in monedas:

 while m >= moneda:

 m = m - moneda

 cantidad = cantidad + 1

return cantidad

Problema No. 2

Knapsack problem fraccionado: el problema visto en clase de un ladrón con una bolsa que aguanta W unidades u de peso, y busca robar el máximo valor que pueda de una selección de n artículos, donde el artículo i tiene precio p_i y hay w_i u 's disponibles del mismo. No es necesario robar el artículo completo; es posible tomar porciones del mismo (a nivel de u 's).

a. Propiedades que posee el problema.

- Subestructura óptima
- Greedy-choice property
- Matroide ponderada

b. Demostración de las propiedades.

Subestructura óptima:

Sea $OPT(W)$ el valor máximo que puede obtenerse con capacidad W . Suponemos que una solución óptima toma una cantidad x_i de un artículo i . Entonces queda una capacidad restante $W - x_i$. Si la solución para esa capacidad restante no fuera óptima, podría reemplazarse por una mejor y así obtener una solución total mejor, esto genera una contradicción. Por lo tanto, el problema posee subestructura óptima.

Greedy-choice property

Se define la relación valor-peso de un artículo como $\frac{p_i}{w_i}$. La estrategia greedy consiste en elegir primero el artículo con mayor relación valor-peso. Suponemos que una solución óptima toma parte de un artículo con mejor relación valor-peso antes que uno con mayor relación. Entonces podemos reemplazar una pequeña parte del artículo peor por el mejor, manteniendo el mismo peso pero aumentando el valor total, lo que genera una contradicción. Por lo tanto, al elegir primero el artículo con mayor relación valor-peso se puede decir que es una elección segura.

Matroide ponderada

La restricción del problema es no superar la capacidad $\sum x_i \leq W$ y el objetivo es maximizar el valor total. Como los artículos pueden dividirse, las decisiones se toman independientemente sobre cada unidad de peso. La solución óptima se obtiene realizando elecciones greedy sucesivas. Por lo tanto, el problema puede modelarse mediante una estructura tipo matroide ponderada.

c. Pseudocódigo

KnapsackFraccionado(W, articulos)

Ordenar articulos de mayor a menor según valor-peso

valorTotal = 0

for articulo in articulos:

 if W == 0:

 break

 if articulo.peso <= W:

 tomar articulo completo

 W = W – articulo.peso

 valorTotal = valorTotal + articulo.valor

 else

 fraccion = W / articulo.peso

 valorTotal = valorTotal + (articulo.valor * fraccion)

 W = 0

Return valorTotal

Problema No. 3

Dado un número entero positivo n y el teclado de un Nokia 3230, que tiene dígitos del 0 al 9 asociados con cada tecla; se desea contar la cantidad total de combinaciones de n dígitos que existen, bajo las siguientes restricciones:

- a. Podemos comenzar con cualquier dígito.
- b. El siguiente teclazo debe ser arriba, abajo, a la izquierda o a la derecha del teclazo más reciente.
- c. El teclado también tiene las teclas * y # que no es permitido presionar.

a. Propiedades que posee el problema.

- Subproblemas traslapados

b. Demostración de las propiedades.

Subproblemas traslapados:

Para calcular la cantidad total de combinaciones de longitud n , se necesita calcular $C(d, k)$ para varios dígitos d y longitudes k . Un mismo subproblema $C(d, k)$ puede ser usado varias veces al calcular combinaciones más largas. Por ejemplo, la cantidad de formas que terminan en un dígito d con longitud k puede contribuir al cálculo de varios dígitos vecinos en longitud $k + 1$. Por lo tanto, el problema posee subproblemas traslapados.

c. Pseudocódigo

combinacionesNokia(n)

vecinos[0] = [0,8]

vecinos[1] = [1,2,4]

vecinos[2] = [2,1,3,5]

vecinos[3] = [3,2,6]

vecinos[4] = [4,1,5,7]

Jose Ordoñez - 231329

vecinos[5] = [5,2,4,6,9]

vecinos[6] = [6,3,5,9]

vecinos[7] = [7,4,8]

vecinos[8] = [8,5,7,9,0]

vecinos[9] = [9,6,8]

dp = arreglo de tamaño 10 lleno de 1

for longitud from (2,n):

 nuevo = arreglo de tamaño 10 lleno de 0

 for digito from (0,9)

 for vecino in vecinos[digito]:

 nuevo[digito] = nuevo[digito] + dp[vecino]

 dp = nuevo

total = suma de los valores de dp

return total